Native Markdown Rendering

This document is a compact fixture for the renderer. It checks the typographic rhythm between headings, paragraph text, inline code like `LayoutLine`, inline math like $E = mc^2$, links such as the Rust project, tables, images, code blocks, and display math.

Spacing System

The page uses a quiet serif body face, a stronger sans heading face, and a modular scale. Paragraph leading is intentionally generous without drifting into double-spaced territory. The result should feel like a designed technical note rather than a browser printout.

Inline code should sit inside the line without throwing off the baseline. Links are colored and underlined, while math remains visually distinct: $a^2 + b^2 = c^2$.

Notes, Lists, and Citations

> **NOTE**
> Callouts use GitHub-style note syntax and render as native PDF blocks, not HTML boxes. They can include **strong text**, *emphasis*, inline code like `RenderOptions`, and citations like *(Knuth 1984)*.

- Plain bullet markers use native shapes and align with the list text.
- Citation markers such as *(Lamport 1994)* are styled inline until bibliography resolution exists.

☑ Task list markers keep their checked state.

☐ Plain list layout shares the same body rhythm as paragraphs.

1. Ordered list markers use native numbering instead of browser text layout.
2. Continuation lines stay aligned with the ordered list body text.

Footnote references render inline[1].

1. Footnotes are parsed from standard GitHub-style Markdown definitions and rendered in compact note text.

Code Blocks

RUST

```rust
fn wrap_line(words: &[&str], max_width: f32) -> Vec<String> {
    let mut lines = Vec::new();
    let mut current = String::new();

    for word in words {
        if current.len() + word.len() + 1 > max_width as usize {
            lines.push(current);
            current = String::new();
        }
        current.push_str(word);
        current.push(' ');
    }

    lines.push(current.trim_end().to_owned());
    lines
}
```

PYTHON

```python
def accent_for(language: str) -> str:
    return {"python": "blue", "rust": "red"}.get(language, "slate")
```

Display Math

$$\int_0^1 x^2 dx = \frac{1}{3}$$

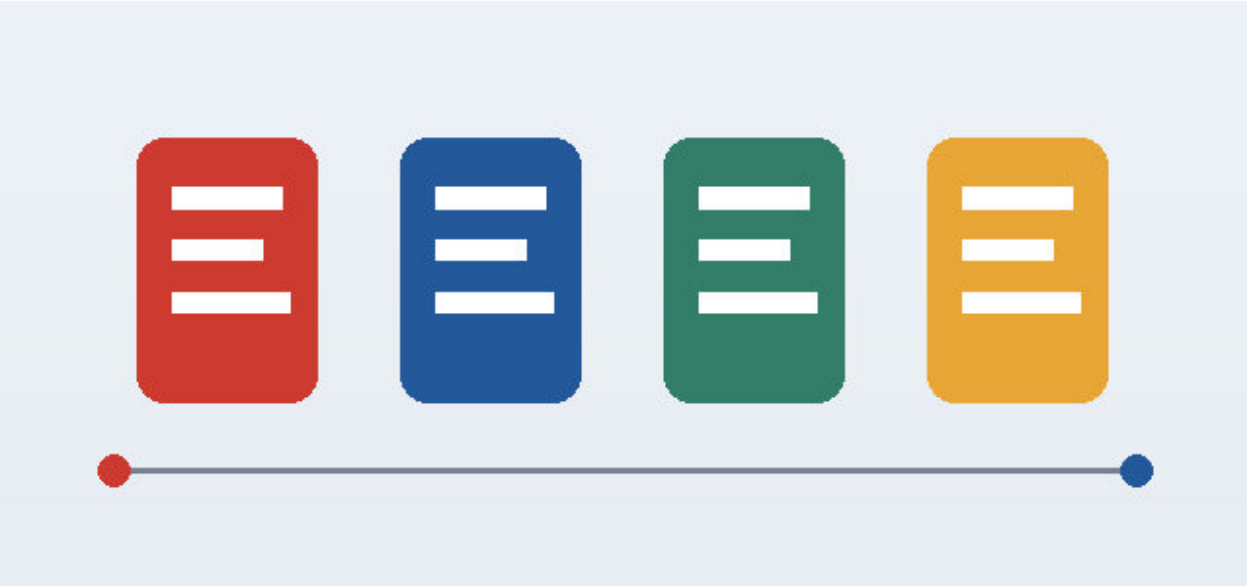
$$\binom{n}{k} = \frac{n!}{k!\,(n - k)!}$$

Math is rendered through PageTeX directly into native PDF text and rule primitives, so formulas remain selectable instead of being flattened into browser output. The parser keeps inline and display math as distinct nodes, so layout, pagination, tables, and image handling stay independent from the math backend.

Tables

Feature	Rendering choice	Status
Headings	Golden-ratio-ish modular scale with measured spacing	Done
Inline code	Monospace with subtle background and padding	Done
Tables	Native PDF lines, fills, and wrapped cell text	Done
Images	Embedded through PDF image XObjects	Done

Images



A structured document pipeline diagram rendered as a bitmap fixture.

Smaller Heading

The last paragraph checks subheading spacing after media. It should land cleanly, with enough top space to signal a new section and enough bottom space to return to normal reading rhythm.